Лабораторная работа № 4

Вариант Е-2

При выполнении лабораторной работы не допускается использовать сторонние библиотеки, за исключением `csv`, `json`, `random`, `re`, `typing`.

Задание 1

1. Загрузите файл `people_cards.json`. Реализуйте классы, необходимые для представления информации из файла в программе.
2. Реализуйте метод строкового представления объекта класса для каждого из реализованных классов.
3. Для каждого из реализованных методов, представляющих действия ниже, помимо выполнения операций также необходимо осуществлять вывод на экран информации о происходящем событии.
4. Каждый человек (и взрослый, и ребенок) может совершить покупку некоего товара на сумму, не превышающую баланс карты (в случае ребенка – также не превышающую лимит по карте). При покупке товара должна выводится информация о том, кто совершает покупку (фамилия, имя, возраст (если это ребенок), название товара и сумма покупки).
5. Родитель может снять наличные деньги со своей карты и передать их своему ребенку, который, в свою очередь, пополняет одну из своих карт на указанную сумму.
6. Каждый человек (и взрослый, и ребенок) может открыть накопительный счет, который хранит в себе накопленную сумму (начальное значение – 0, если человек не открыл счет с пополнением на некоторую сумму). Также у счета может быть указан процент (0 и выше), который будет отчисляться на указанный счет со всех поступлений человека. Человек может пополнять счет со своей карты (на положительную сумму) или снимать деньги со счета на карту (положительную сумму).
7. Взрослые могут переводить деньги другим взрослым по номеру карты.
8. Взрослые могут получать заработную плату.
9. Взрослые могут создать инвестиционный счет, у которого есть текущий баланс. Его можно пополнить с карты (на положительную сумму), вывести деньги с него на карту (положительную сумму). Имеется возможность обновить состояние инвестиционного счета, в процессе обновления баланс счета изменяется на случайную величину (положительную или отрицательную (`random.randint(-500,500)`). В

случае, если на инвестиционном счете баланс стал отрицательным, счет блокируется. В то время, как счет находится в заблокированном состоянии, человеку доступна только операция пополнения счета. Счет остается заблокированным, пока баланс отрицательный.

10. Взрослый может создать обязательный платеж по одной из своих карт, у которого указано название и сумма. Соответственно, человек может совершить данный обязательный платеж с изменением баланса карты.
11. В виде отдельных модулей должны быть реализованы накопительный и инвестиционный счет (один файл), банковские карты (один файл), люди (один файл). В основном модуле необходимо осуществить импорт всех данных из файла, полностью преобразовать их к объектам соответствующих классов и продемонстрировать работу всех описанных методов.
12. В каждом отдельном модуле необходимо указать демонстрационный код (если возможно продемонстрировать работу методов в пределах данного модуля), демонстрационный код должен выполняться только в случае запуска модуля напрямую без импортирования в другой модуль.
13. Продемонстрировать работу реализованных методов на каких-либо отдельно взятых сущностях из набора импортированных данных. Демонстрация должна иллюстрировать все возможные возникающие ситуации.

globals.py

```
people = {}
cards = {}
saveing_accounts = {}
investment_accounts = {}
```

cards.py

```
from accounts import Account



class Card(Account):
    def __init__(self, number, balance, owners_id, limit=None):
        super().__init__(owners_id, balance)
        self.number = number
        self.limit = limit
        self.mandatory_payments = []

    def __str__(self):
        return f'Карта {self.number}: баланс {self.balance}' + (
            f', лимит: {self.limit}' if self.limit is not None else '')

    def can_pay(self, price):
        if self.limit is not None:
            return price <= self.balance and price <= self.limit
        return price <= self.balance

    def pay(self, price):
        if self.can_pay(price):
            self.balance -= price
            if self.limit is not None:
                self.limit -= price
            return True
        return False

    def Deposit(self, amount):
        from globals import saveing_accounts
        if saveing_accounts.get(self.owners_id) is not None:
            percentage = saveing_accounts.get(self.owners_id).percentage
            bonus = amount * (percentage / 100)
            saveing_accounts.get(self.owners_id).Deposit(bonus, self)
        self.balance += amount


class Mandatory_payment:
    def __init__(self, name, amount, card_number):
        if amount <= 0:
            raise ValueError("Сумма платежа должна быть положительной")

        self.name = name
        self.amount = amount
        self.card_number = card_number
        self.paid = False
```

```
    def __str__(self):
        status = "Оплачен" if self.paid else "Ожидает оплаты"
        return f"    Обязательный платеж '{self.name}':      {self.amount}руб. ({status})"

    def pay(self):
        from globals import cards
        if self.paid:
            return False, f"    Платеж '{self.name}' уже оплачен"

        self.paid = True
        cards[self.card_number].pay(self.amount)
        print(f"    Платеж '{self.name}' на сумму {self.amount}руб. выполнен")
        print(f"    Новый баланс карты:       {round(cards[self.card_number].balance, 1)}")

    def reset(self):
        self.paid = False
        print(f"Статус платежа '{self.name}' сброшен")


    def transfer_from_card_to_card(from_card, to_card, amount):
        print(f'Перевод с карты: {from_card.number} на карту:  {to_card.number} {amount}
руб.')

        if amount <= 0:
            print('    Сумма перевода должна быть положительной.')
            return False

        if amount > from_card.balance or (from_card.limit is not None and amount >
from_card.limit):
            print('    На карте недостаточно средств или превышен лимит.')
            return False

        if from_card.can_pay(amount):
            from_card.balance -= amount
            if from_card.limit is not None:
                from_card.limit -= amount

        to_card.balance += amount

        print('    Перевод успешен')
        print(f'    С карты {from_card.number} переведено {amount}руб. на карту
{to_card.number}, остаток {from_card.balance}')
```

```python
if __name__ == "__main__":
    print("Демонстрация работы карт:")

    card1 = Card("1111222233334444", 1000, 1)
    print(f"Создана карта: {card1}")

    card2 = Card("5555666677778888", 500, 2, limit=300)
    print(f"Создана карта: {card2}")

    print("\nТест перевода между картами:")
    transfer_from_card_to_card(card1, card2, 200)

    print(f"\nКарта 1 после перевода: {card1}")
    print(f"Карта 2 после перевода: {card2}")

    print("\nТест обязательного платежа:")
    payment = Mandatory_payment("Интернет", 150, card1.number)
    print(payment)
```

```
Демонстрация работы карт:
Создана карта: Карта 1111222233334444: баланс 1000
Создана карта: Карта 5555666677778888: баланс 500, лимит: 300

Тест перевода между картами:
Перевод с карты: 1111222233334444 на карту: 5555666677778888 200руб.
    Перевод успешен
    С карты 1111222233334444 переведено 200руб. на карту 5555666677778888, остаток 800

Карта 1 после перевода: Карта 1111222233334444: баланс 800
Карта 2 после перевода: Карта 5555666677778888: баланс 700, лимит: 300

Тест обязательного платежа:
    Обязательный платеж 'Интернет': 150руб. (Ожидает оплаты)
```

accounts.py

```
import random

class Account:
    next_id = 1

    def __init__(self, owners_id, balance=0.0):
        self.balance = balance
        self.owners_id = owners_id
        self.id = Account.next_id
        Account.next_id += 1

    def Deposit(self, amount, card):
        if type(amount) != int and type(amount) != float or amount < 0:
            print(f'    Сумма должна быть положительным числом')
            return False
        if card.can_pay(amount):
            self.balance += amount
            print(f'    С карты {card.number} переведено {amount} руб. на счёт')
            print(f"    Общий баланс счёта: {self.balance}руб.")
        self.balance += amount


class Saveing_account(Account):
    def __init__(self, owners_id):
        super().__init__(owners_id)
        self.percentage = 9

    def __str__(self):
        return f'Накопительный счёт {self.id}, баланс: {self.balance}, процентная ставка:
{self.percentage}%'

    def withdraw_to_card(self, amount, card):
        if type(amount) != int and type(amount) != float or amount < 0:
            print(f'    Сумма снятия должна быть положительным числом')
            return False
        if amount > self.balance:
            print(f"    Недостаточно средств на накопительном счёте")
            return False
        self.balance -= amount
        card.balance += amount
        print(f"    С накопительного счёта снято {amount} руб. на карту {card.number}")
        print(f"    Остаток на накопительном счёте: {self.balance} руб.")


class Investment_account(Account):
    def __init__(self, owners_id):
        super().__init__(owners_id)
        self.blocked = False
```

```
    def withdraw_to_card(self, amount, card):
        if self.blocked:
            print("    Счёт заблокирован, вывод невозможен")
            return False
        if type(amount) != int and type(amount) != float or amount < 0:
            print(f'    Сумма снятия должна быть положительным числом')
            return False
        if amount > self.balance:
            print(f"    Недостаточно средств на инвестиционном счёте")
            return False
        self.balance -= amount
        card.balance += amount
        print(f"    С Инвестиционного счёта снято {amount} руб. на карту {card.number}")
        print(f"    Остаток на Инвестиционном счёте: {self.balance} руб.")

    def update_balance(self):
        change = random.randint(-500, 500)
        self.balance += change
        print(f"    Инвестиционный счёт обновлён. Изменение: {change}руб., новый баланс:
{self.balance}руб.")
        if self.balance < 0:
            self.blocked = True
            print("    Счёт заблокирован из-за отрицательного баланса")
        elif self.blocked and self.balance >= 0:
            self.blocked = False
            print("    Счёт разблокирован")

if __name__ == "__main__":
    print("Демонстрация работы счетов:")

    test_savings = Saveing_account(1)
    print(test_savings)

    test_investment = Investment_account(1)
    print(test_investment)

    for i in range(3):
        print(f"\nОбновление {i + 1}:")
        test_investment.update_balance()
```

```
Демонстрация работы счетов:
Накопительный счёт 1, баланс: 0.0, процентная ставка: 9%
Инвестиционный счёт 2, баланс: 0.0руб., статус: активен

Обновление 1:
    Инвестиционный счёт обновлён. Изменение: 60руб., новый баланс: 60.0руб.

Обновление 2:
    Инвестиционный счёт обновлён. Изменение: 280руб., новый баланс: 340.0руб.

Обновление 3:
    Инвестиционный счёт обновлён. Изменение: -131руб., новый баланс: 209.0руб.
```

people.py

```
from accounts import Saveing_account, Investment_account



class Person:
    next_id = 1

    def __init__(self, first_name, last_name, age=None):
        self.id = Person.next_id
        Person.next_id += 1
        self.first_name = first_name
        self.last_name = last_name
        self.age = age
        self.cash = 0

    def __str__(self):
        from main import get_cards  # Импортируем здесь, чтобы избежать циклического
импорта
        cards_info = f', число карт: {len(get_cards(self.id))}'
        return f'{self.last_name} {self.first_name}{cards_info}'

    def pakaji_karti(self):
        from main import get_cards
        if not get_cards(self.id):
            print(f'    У {self.last_name} {self.first_name} нет карт')
            return False
        for i in range(len(get_cards(self.id))):
            limit_info = f', лимит: {get_cards(self.id)[i].limit}руб.' if get_cards(self.id)[i].limit is not
None else ''
            print(
                f'{i + 1}. {get_cards(self.id)[i].number} --- баланс: {get_cards(self.id)[i].balance}руб.
{limit_info}')

    def buy(self, product_name, price):
        from main import get_cards
        for card in get_cards(self.id):
            if card.can_pay(price):
                card.pay(price)
                print(f'    {self.last_name} {self.first_name} покупает "{product_name}" за {price}
руб.')
                print(f'    Использовалась карта: {card.number}')
                print(f'    Осталось на карте: {card.balance}руб.')
                if card.limit is not None:
                    print(f'Оставшийся лимит: {card.limit}')
                return True
        print(f'Недостаточно средств или превышен лимит')
        return False
```

```
    def Deposit_to_card(self, card_index, amount):
        from main import get_cards
        if amount is None:
            amount = self.cash
        if amount > self.cash:
            print('Не достаточно средств')
            return False
        if 0 <= card_index < len(get_cards(self.id)):
            get_cards(self.id)[card_index].deposit(amount)
            self.cash -= amount
            print(f'Карта: {get_cards(self.id)[card_index].number} пополнена на {amount}')
            return True
        return False

    def receive_calary(self, to_card_number, amount=int):
        from globals import cards
        if cards.get(to_card_number) is None:
            print(f'Нет карты с номером {to_card_number}')
            return False
        if self.age is not None:
            if self.age < 18:
                print('У детей нет зарплаты')
                return False
        print(f'Получение {self.last_name} {self.first_name} зарплаты:')
        if type(amount) != int and type(amount) != float or amount < 0:
            print(f'Сумма зарплаты должна быть положительным числом')
            return False
        cards[to_card_number].balance += amount
        print('    Получена зарплата')
        print(f'    Новый балланс карты {to_card_number}: {cards[to_card_number].balance}
руб.')

    def create_mandatory_payment(self, name, amount, card_index):
        from main import get_cards
        from globals import cards
        card_list = get_cards(self.id)
        if card_index < len(card_list):
            card_number = card_list[card_index].number
            if card_number in cards:
                card = cards[card_number]
                from cards import Mandatory_payment
                card.mandatory_payments.append(Mandatory_payment(name, amount,
card_number))
                print('Обязательный платёж создан')
            else:
                print('Такой карты нет')
        else:
            print('Нет карты с таким индексом')
```

```
def create_savings_account(self):
    from globals import saveing_accounts
    print(f'Создание накопительного счёта для {self.last_name} {self.first_name}:')

    if self.age is not None:
        if self.age < 18:
            print('Детям нельзя создавать счета')
            return False

    if self.id in saveing_accounts:
        print(f'    У {self.last_name} {self.first_name} уже есть накопительный счёт')
        existing_account = saveing_accounts[self.id]
        print(f'    Существующий счёт: {existing_account}')

    savings_account = Saveing_account(self.id)

    saveing_accounts[self.id] = savings_account

    print(f'    Накопительный счёт успешно создан!')
    print(f'    ID счёта: {savings_account.id}')
    print(f'    Процентная ставка: {savings_account.percentage}%')
    print(f'    Начальный баланс: {savings_account.balance}руб.')

def create_investment_account(self):
    from globals import investment_accounts
    print(f'Создание инвестиционного счёта для {self.last_name} {self.first_name}:')

    if self.age is not None:
        if self.age < 18:
            print('Детям нельзя создавать счета')
            return False

    if self.id in investment_accounts:
        print(f'    У {self.last_name} {self.first_name} уже есть инвестиционный счёт')
        existing_account = investment_accounts[self.id]
        print(f'    Существующий счёт: {existing_account}')

    investment_account = Investment_account(self.id)

    investment_accounts[self.id] = investment_account

    print(f'    Инвестиционный счёт успешно создан!')
    print(f'    ID счёта: {investment_account.id}')
    print(f'    Начальный баланс: {investment_account.balance}руб.')
    print(f'    Статус: {"заблокирован" if investment_account.blocked else "активен"}')
```

```
class Child(Person):
    def __init__(self, first_name, last_name, age, parent_id):
        super().__init__(first_name, last_name, age)
        self.parent_id = parent_id

    def __str__(self):
        from main import get_cards
        cards_info = f', число карт: {len(get_cards(self.id))}'
        return f'{self.last_name} {self.first_name}, {self.age} лет{cards_info}'

    def add_cash(self, amount):
        self.cash += amount

class Parent(Person):
    def __init__(self, first_name, last_name, children_ids=None):
        super().__init__(first_name, last_name)
        self.children_ids = children_ids or []

    def __str__(self):
        from main import get_cards
        cards_info = f', число карт: {len(get_cards(self.id))}'
        children_info = f', число детей: {len(self.children_ids)} детей'
        return f'{self.last_name} {self.first_name}{children_info}{cards_info}'

    def add_child(self, child_id: int):
        self.children_ids.append(child_id)

    def give_cash_to_child(self, amount, child, from_card_index):
        from main import get_cards
        if type(amount) != int and type(amount) != float or amount < 0:
            print(f'Сумма должна быть положительным числом')
            return False
        if child.id not in self.children_ids:
            print(f'{child.last_name} {child.first_name} не является ребёнком {self.last_name}
{self.first_name}')
            return False
        if from_card_index >= len(get_cards(self.id)):
            print(f'У {self.last_name} {self.first_name} меньше {from_card_index} карт')
            return False
        from_card = get_cards(self.id)[from_card_index]
        if not from_card.can_pay(amount):
            print(f'Недостаточно средств на карте: {from_card.number} для перевода')
            return False
        from_card.pay(amount)
        child.add_cash(amount)
        print(f'{self.last_name} {self.first_name} передал(а) {child.last_name} {child.first_name}
{amount}руб.')
        print(f'    С карты: {from_card.number}')
```

```python
if __name__ == "__main__":
    print("Демонстрация работы классов людей:")

    parent = Parent("Иван", "Иванов")
    print(f"Создан родитель: {parent}")

    child = Child("Анна", "Иванова", 12, parent.id)
    print(f"Создан ребенок: {child}")

    parent.add_child(child.id)
    print(f"Родитель {parent.last_name} имеет детей: {parent.children_ids}")

    print("\n Попытка создать накопительный счет:")
    savings = parent.create_savings_account()

    print("\n Попытка создать инвестиционный счет:")
    investment = parent.create_investment_account()

    print("\n Ребенок пытается создать накопительный счет:")
    child_savings = child.create_savings_account()

    print("\n Ребенок пытается создать инвестиционный счет:")
    child_investment = child.create_investment_account()
```

```
Демонстрация работы классов людей:
Создан родитель: Иванов Иван, число детей: 0 детей, число карт: 0
Создан ребенок: Иванова Анна, 12 лет, число карт: 0
Родитель Иванов имеет детей: [2]

 Попытка создать накопительный счет:
Создание накопительного счёта для Иванов Иван:
    Накопительный счёт успешно создан!
    ID счёта: 1
    Процентная ставка: 9%
    Начальный баланс: 0.0руб.

 Попытка создать инвестиционный счет:
Создание инвестиционного счёта для Иванов Иван:
    Инвестиционный счёт успешно создан!
    ID счёта: 2
    Начальный баланс: 0.0руб.
    Статус: активен
```

```
 Ребенок пытается создать накопительный счет:
Создание накопительного счёта для Иванова Анна:
Детям нельзя создавать счета

 Ребенок пытается создать инвестиционный счет:
Создание инвестиционного счёта для Иванова Анна:
Детям нельзя создавать счета

 Взрослый пытается создать еще один накопительный счет:
Создание накопительного счёта для Иванов Иван:
    У Иванов Иван уже есть накопительный счёт
    Существующий счёт: Накопительный счёт 1, баланс: 0.0, процентная ставка: 9%
    Накопительный счёт успешно создан!
    ID счёта: 3
    Процентная ставка: 9%
    Начальный баланс: 0.0руб.
```

```
import json
from people import Person, Child, Parent
from cards import Card, Mandatory_payment, transfer_from_card_to_card
from accounts import Saveing_account, Investment_account
from globals import people, cards, saveing_accounts, investment_accounts



def get_cards(person_id):
    persons_cards = []
    for card in list(cards.values()):
        if card.owners_id == person_id:
            persons_cards.append(card)
    return persons_cards


def transfer(from_card_number, to_card_number, amount):
    if people.get(cards.get(from_card_number).owners_id).age is not None:
        if people.get(cards.get(from_card_number).owners_id).age <= 18:
            print('Детям нельзя переводить деньги')
            return False
    if people.get(cards.get(to_card_number).owners_id).age is not None:
        if people.get(cards.get(to_card_number).owners_id).age <= 18:
            print('Детям нельзя переводить деньги')
            return False
    if cards.get(from_card_number, False) and cards.get(to_card_number, False):
        transfer_from_card_to_card(cards[from_card_number], cards[to_card_number],
amount)
```

```
def load_from_json(file_p):
    with open(file_p, 'r', encoding='utf-8') as file:
        data = json.load(file)

    for person_data in data:
        parent = Parent(first_name=person_data.get('first_name'),
last_name=person_data.get('last_name'))
        people[parent.id] = parent

        for card_data in person_data.get('cards', []):
            card = Card(number=card_data.get('number'), balance=card_data.get('balance'),
limit=card_data.get('limit'), owners_id=parent.id)
            cards[card.number] = card

        for child_data in person_data.get('children', []):
            child = Child(first_name=child_data.get('first_name'),
last_name=child_data.get('last_name'), age=child_data.get('age'), parent_id=parent.id)
            people[child.id] = child
            people[child.parent_id].children_ids.append(child.id)

            for child_card_data in child_data.get('cards', []):
                child_card = Card(number=child_card_data.get('number'),
balance=child_card_data.get('balance'), limit=child_card_data.get('limit'), owners_id=child.id)
                cards[child_card.number] = child_card

    return [people, cards]


if __name__ == "__main__":
    load_from_json('people_cards.json')
    print(f'Загружено {len(people)} человек(а)')

    s_parent = None
    for person in list(people.values()):
        if get_cards(person.id) and isinstance(person, Parent):
            if person.children_ids:
                for ids in person.children_ids:
                    if len(get_cards(ids)) >= 2:
                        s_parent = person
                        break
                if s_parent:
                    break
```

```
if not s_parent:
    print('Таких нет')

parent = s_parent
child = people[parent.children_ids[0]]

print(f'Родитель: {parent}')
print(f'Ребёнок: {child}\n')

print('X' * 77)

print('Карты:\n')
parent.pakaji_karti()
print()
print('X' * 77)
print('Покупка родителем:\n')

parent.buy('Жавачка', 1)
print()
parent.buy('Ракета', 99999)
print()

print('X' * 77)
print('Покупка ребёнком:\n')

child.buy('Водичка', 60)
print()
child_cards = get_cards(child.id)
if child_cards:
    card = child_cards[0]
    if card.limit:
        child.buy('Велик', card.limit + 100)
        print()

print('X' * 77)
print('Передача денег ребёнку:\n')
print('Карты до перевода:\n')
print('Родитель:')
parent.pakaji_karti()
print('Ребёнок:')
child.pakaji_karti()
print()

parent.give_cash_to_child(15, child, 0)
print()

parent.give_cash_to_child(9999999, child, 0)

print()
print('X' * 77)
print()
```

```
parent_cards = get_cards(parent.id)
child_cards = get_cards(child.id)

if len(parent_cards) > 1 and child_cards:
    print('Перевод с карты на карту:\n')
    transfer(parent_cards[1].number, child_cards[0].number, 50)

print()
print('X' * 77)
print()

if parent_cards and len(parent_cards) > 1:
    parent.receive_calary(parent_cards[1].number, 500)

print()
print('X' * 77)
print()

if parent_cards and len(parent_cards) > 1:
    parent.create_mandatory_payment('internet', 200, 1)
    if parent_cards[1].mandatory_payments:
        parent_cards[1].mandatory_payments[0].pay()

print('1. ВЗРОСЛЫЙ создает счета:')
print('X' * 77)

print('\na) Создание накопительного счета (10%):')
parent.create_savings_account()

print('\nб) Создание инвестиционного счета:')
parent.create_investment_account()

print('\nв) Попытка создать дубликаты счетов:')
print('   Накопительный счет:')
parent.create_savings_account()
print('   Инвестиционный счет:')
parent.create_investment_account()
print('\n\n2. РЕБЁНОК пытается создать счета:')
print('X' * 77)

print('\na) Создание накопительного счета:')
child.create_savings_account()

print('\nб) Создание инвестиционного счета:')
child.create_investment_account()

print('\n\n3. РАБОТА СО СЧЕТАМИ:')
print('X' * 40)


parent_savings = saveing_accounts.get(parent.id)
parent_investment = investment_accounts.get(parent.id)
```

```
if parent_savings:
    print(f'\nа) Накопительный счет родителя: {parent_savings}')

    parent_cards = get_cards(parent.id)
    if parent_cards:
        print(f'\nб) Пополнение карты родителя (тест начисления процентов):')
        print(f'   Баланс накопительного счета до: {parent_savings.balance}руб.')
        print(f'   Баланс карты до: {parent_cards[0].balance}руб.')

        parent_cards[0].Deposit(1000)

        print(f'   Баланс накопительного счета после: {parent_savings.balance}руб.')
        print(f'   Баланс карты после: {parent_cards[0].balance}руб.')

if parent_investment:
    print(f'\nв) Инвестиционный счет родителя: {parent_investment}')

    print(f'\nг) Обновление инвестиционного счета:')
    parent_investment.update_balance()
    print(f'   Новый баланс: {parent_investment.balance}руб.')
    print(f'   Статус: {"заблокирован" if parent_investment.blocked else "активен"}')
```

```
Загружено 194 человек(а)
Родитель: Feaveryear Kiley, число детей: 1 детей, число карт: 2
Ребёнок: Antram Denise, 19 лет, число карт: 2

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Карты:

1. 3570363825074491 --- баланс: 80.92руб.
2. 3554164416385571 --- баланс: 99.3руб.

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Покупка родителем:

    Feaveryear Kiley покупает "Жавачка" за 1руб.
    Использовалась карта: 3570363825074491
    Осталось на карте: 79.92руб.

Недостаточно средств или превышен лимит

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Покупка ребёнком:

    Antram Denise покупает "Водичка" за 60руб.
    Использовалась карта: 3531603210381292
    Осталось на карте: 86.80000000000001руб.
Оставшийся лимит: 580.95

Недостаточно средств или превышен лимит

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

```
Передача денег ребёнку:

Карты до перевода:

Родитель:
1. 3570363825074491 --- баланс: 79.92руб.
2. 3554164416385571 --- баланс: 99.3руб.
Ребёнок:
1. 3531603210381292 --- баланс: 86.80000000000001руб., лимит: 580.95руб.
2. 3535342601984721 --- баланс: 590.94руб., лимит: 650.46руб.

Feaveryear Kiley передал(а) Antram Denise 15руб.
    С карты: 3570363825074491

Недостаточно средств на карте: 3570363825074491 для перевода

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Перевод с карты на карту:

Перевод с карты: 3554164416385571 на карту: 3531603210381292 50руб.
    Перевод успешен
    С карты 3554164416385571 переведено 50руб. на карту 3531603210381292, остаток 49.3

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Получение Feaveryear Kiley зарплаты:
    Получена зарплата
    Новый балланс карты 3554164416385571: 549.3руб.

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Обязательный платёж создан
    Платеж 'internet' на сумму 200руб. выполнен
    Новый баланс карты: 349.3
```

```
1. ВЗРОСЛЫЙ создает счета:
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

а) Создание накопительного счета (10%):
Создание накопительного счёта для Feaveryear Kiley:
    Накопительный счёт успешно создан!
    ID счёта: 204
    Процентная ставка: 9%
    Начальный баланс: 0.0руб.

б) Создание инвестиционного счета:
Создание инвестиционного счёта для Feaveryear Kiley:
    Инвестиционный счёт успешно создан!
    ID счёта: 205
    Начальный баланс: 0.0руб.
    Статус: активен

в) Попытка создать дубликаты счетов:
   Накопительный счет:
Создание накопительного счёта для Feaveryear Kiley:
    У Feaveryear Kiley уже есть накопительный счёт
    Существующий счёт: Накопительный счёт 204, баланс: 0.0, процентная ставка: 9%
    Накопительный счёт успешно создан!
    ID счёта: 206
    Процентная ставка: 9%
    Начальный баланс: 0.0руб.
   Инвестиционный счет:
Создание инвестиционного счёта для Feaveryear Kiley:
    У Feaveryear Kiley уже есть инвестиционный счёт
    Существующий счёт: Инвестиционный счёт 205, баланс: 0.0руб., статус: активен
    Инвестиционный счёт успешно создан!
    ID счёта: 207
    Начальный баланс: 0.0руб.
    Статус: активен
2. РЕБЁНОК пытается создать счета:
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

а) Создание накопительного счета:
Создание накопительного счёта для Antram Denise:
    Накопительный счёт успешно создан!
    ID счёта: 208
    Процентная ставка: 9%
    Начальный баланс: 0.0руб.

б) Создание инвестиционного счета:
Создание инвестиционного счёта для Antram Denise:
    Инвестиционный счёт успешно создан!
    ID счёта: 209
    Начальный баланс: 0.0руб.
    Статус: активен
```

```
3. РАБОТА СО СЧЕТАМИ:
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

а) Накопительный счет родителя: Накопительный счёт 206, баланс: 0.0, процентная ставка: 9%

б) Пополнение карты родителя (тест начисления процентов):
   Баланс накопительного счета до: 0.0руб.
   Баланс карты до: 64.92руб.
   Баланс накопительного счета после: 90.0руб.
   Баланс карты после: 1064.92руб.

в) Инвестиционный счет родителя: Инвестиционный счёт 207, баланс: 0.0руб., статус: активен

г) Обновление инвестиционного счета:
    Инвестиционный счёт обновлён. Изменение: 242руб., новый баланс: 242.0руб.
   Новый баланс: 242.0руб.
   Статус: активен
```